Problem Understanding and Objective

Clinical Document Intelligence Hub · *Arman*

The brief. Clinical and administrative staff read intake forms, discharge summaries, lab reports and physician notes by hand to pull out the same handful of facts. It is slow, inconsistent, and error-prone.

Where I think the real problem sits. Extraction accuracy is not the hard part. Trust is. A structured field with no provenance still forces the reviewer to re-read the document to check it — which is exactly the manual work we claimed to remove. And a field that is confidently wrong is worse than one that is missing.

Objective. Document → verified structured record → triaged action. Every field carries a confidence and the source span it came from. Anything that cannot be verified is withheld and routed to a human, never displayed as though it were checked.

18 fields extracted per document

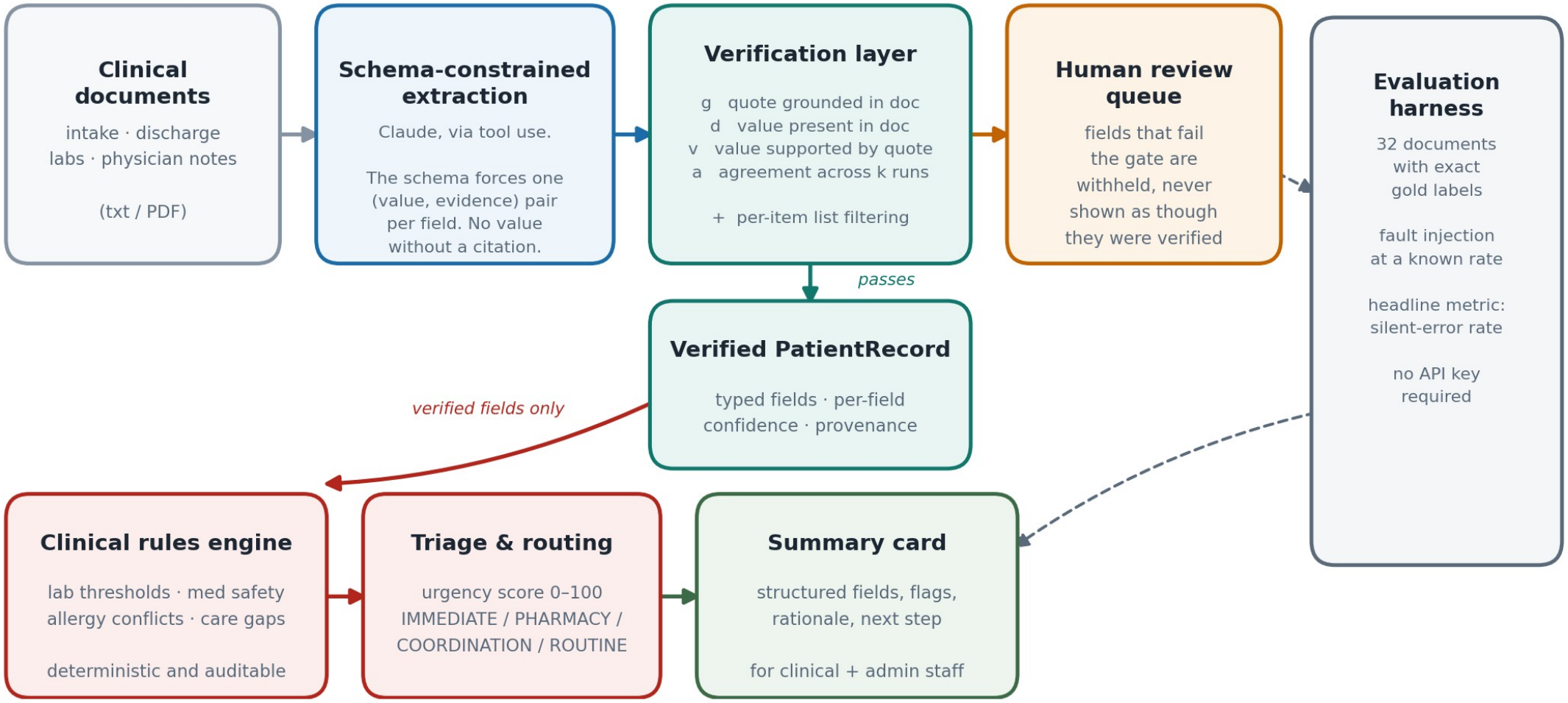
4 document types supported

32 document corpus, exact gold labels

21 clinical rules, every flag traceable

Solution Architecture and Design Flow

The LLM extracts. Verification gates. Rules decide. Each stage does only what it is actually reliable at.



LLM Reading prose → fields

Verification Trusting a value

Rules Escalating a patient

Implementation Highlights

Confidence is computed from measurable signals, never asked for — verbalised LLM confidence is poorly calibrated.

Four signals behind every field

- **g** **grounding —** is the evidence quote really in the document?
- **d** **presence —** is the value in the document, independent of its quote?
- **v** **support —** is the value inside the quote it cites?
- **a** **agreement —** do k independent samples return the same value?

```
confidence = 0.30·g + 0.25·d + 0.15·v + 0.30·a
```

Measured, not asserted

96.2% — **fabrications caught**
under fault injection (50 / 52)

0.0% — **false block rate**
0 of 143 clean fields withheld

14.0% — **baseline silent error**
regex control arm, for comparison

The gate that does the work

Named entities — patient, diagnosis, drug, allergen — hard-gate on presence. A name that is not in the text is an invention, full stop.

Numerics deliberately do not: 37.6 °C correctly extracts as 99.7 °F, and “one hundred and twelve” as 112. Neither literal appears in the document. Hard-gating those would punish the model for doing the conversion the schema asked for.

Lists are filtered per item. A medication list that is 4/5 correct is not 80% safe — the phantom fifth entry is the entire risk.

cdi/verify.py

```
g = grounding_score(quote, doc)
d = presence_score(value, doc)
v = support_score(value, quote)
a = agreement_score(samples)

fabricated = (g < GATE) or (
    d < GATE and not is_numeric(value))

if fabricated:  UNGROUNDED  # withhold
elif dropped:   ABSTAINED   # list invention
elif conf < T:  ABSTAINED   # low confidence
else:           EXTRACTED   # show it
```

Challenges and Learnings

Three of these were my own bugs, found by measurement rather than by reading the code again.

1

My corpus was marking its own homework

The first eval gave the regex baseline 99.4%. I had written both the templates and the regexes, so they matched by construction. I added 15 free-prose narrative documents — spelled-out numbers, Celsius, no field labels. The baseline fell to 86% and missed 177 fields. A benchmark you author yourself flatters you until you deliberately attack it.

2

The grounding gate had a hole

A fabricated patient name passed verification: the quote “Patient Name : <wrong name>” fuzzy-matched the real line because the boilerplate prefix carried the score. Checking the quote is not the same as checking the value. Adding a direct value-in-document test is what actually catches invention.

3

Correct answers were being rejected

Hard-gating numerics on presence punished the model for doing the unit conversion the schema asked for. The fix was to make the gate asymmetric — hard for named entities, soft for numerics — and to match spelled-out numbers and Celsius equivalents explicitly.

4

You cannot measure a hallucination-catcher with an extractor that cannot hallucinate

The regex baseline only copies or fails, so it showed the verification layer doing nothing. Rather than assert the layer works, I injected synthetic hallucinations at a known rate — real quote, subtly wrong value — and measured a 96.2% catch rate with zero false blocks. No API key needed to reproduce it.

The learning that changed the most: *picking silent error rate over accuracy as the headline metric reshaped the system more than any model choice did. Once the number you optimise punishes confident wrongness rather than silence, abstention stops being a failure and becomes a feature.*

Demo Summary and Next Steps

Worked example — one discharge summary

Four traps in the source document, all handled: prior-visit vitals ignored · family-history condition excluded · discontinued drug excluded · 10 risk flags raised

CRITICAL Amoxicillin prescribed against a documented penicillin allergy

CRITICAL Metformin contraindicated at eGFR 24

HIGH Discharged with no follow-up, on a high-risk diagnosis

→ Routed: IMMEDIATE CLINICAL REVIEW, urgency 100/100

What runs today

```
streamlit run app.py
python -m cdi.summarize <document>
python run_eval.py
python run_eval.py --arm anthropic --k 3
```

The baseline arm, the full evaluation harness and the UI all run with no API key. **The fault-injection result is reproducible on any machine in one command.**

If I had another week

Set the threshold, don't pick it

Fit abstention against an explicit cost ratio for a missed field vs. a wrong one, rather than a hand-chosen 0.70.

A corpus I didn't write

Re-measure on a public de-identified set and report the delta honestly — my numbers are on synthetic data.

Field-level escalation

Route only the low-confidence fields to a stronger model instead of re-running the whole document.

Cross-document reconciliation

Compare an intake form against its discharge summary and flag contradictions between them.

Arman · README, evaluation report and full source in the submitted repository